SCALE FOR PROJECT PYTHON MODULE (/PROJECTS/PYTHON-MODULE-08)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere-v2.42madrid.com:vogsphere/intra-uuid-5caeb369-d2
```

Introduction

- Remain polite, courteous, respectful and constructive
throughout the evaluation process. The well-being of the community
depends on it.

- Work with the person (or the group) being evaluated to identify any potential
dysfunctions in the work. Take the time to discuss
and debate the problems you have identified.

- You must consider that there may be differences in how your
peers have understood the project's instructions and the
scope of its functionalities. Always keep an open mind and grade
them as honestly as possible. The pedagogy is valid
only if peer evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student or group's
Git repository.

- Double-check that the Git repository belongs to the student
or the group. Ensure that the work is for the relevant project
and also check that "git clone" is used in an empty folder.

- Check carefully that no malicious aliases were used to fool you
and make you evaluate something other than the content of the
official repository.

- To avoid any surprises, carefully check that both the evaluating
and the evaluated students have reviewed the possible scripts used
to facilitate the grading.

- If the evaluating student has not completed that particular
project yet, it is mandatory for this student to read the
entire subject prior to starting the defence.

- Use the flags available on this scale to signal an empty repository,
a non-functioning program, a norm error, cheating, etc. In these cases,
the grading is over and the final grade is 0 (or -42 in case of
cheating). However, with the exception of cheating, you are
encouraged to continue to discuss your work (even if you have not
finished it) in order to identify any issues that may have caused
this failure and avoid repeating the same mistake in the future.

- Remember that during the review, no unexpected, premature, uncontrolled
termination of the program should occured; otherwise, the final grade is 0. Use the
appropriate flag.
You should never need to edit any file except the configuration file, if it exists.
If you want to edit a file, take the time to explain the reasons with the
evaluated student and make sure both of you agree to this.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/203238/en.subject.pdf)

Preliminaries

Basics

- Only grade the work that is in the learner's or group's Git repository.
- Check that the Git repository contains the following directories: ex0/, ex1/, ex2/
- Verify that each directory contains the required files as specified in the subject.
- Ensure no .env files with real secrets are committed to the repository.

If any of these conditions are not met, the peer-review stops here.

Yes No

Exercise 0

Exercise 0 - Virtual Environments (The Construct)

Navigate to ex0/ directory and examine the files:

Required files:

- construct.py

Test the construct.py program:

1. Run it outside a virtual environment
2. Create a virtual environment as instructed by the program
3. Activate the virtual environment and run the program again

The program should:

- Correctly detect whether it's running in a virtual environment
- Display different outputs for each scenario
- Provide clear instructions for creating a virtual environment
- Show relevant path information

Ask the learner to explain:

- What is a virtual environment and why is it important?
- What problems do virtual environments solve?
- How does their program detect the virtual environment?

Does the exercise work correctly and does the learner demonstrate understanding?

Yes No

Exercise 1

Exercise 1 - Package Management (Loading Programs)

Navigate to ex1/ directory and examine the files:

Required files:

- loading.py
- requirements.txt
- pyproject.toml

Test the package management:

1. Try running loading.py without dependencies installed
2. Install dependencies using pip (requirements.txt)
3. Test the program functionality
4. If Poetry is available, test Poetry installation (pyproject.toml)

The program should:

- Handle missing dependencies gracefully with helpful error messages
- Use pandas, numpy, and matplotlib as specified
- Generate some form of data analysis or visualization
- Show package version information

Ask the learner to explain:

- The difference between pip and Poetry
- Why dependency management is important
- How their program handles missing dependencies
- How they simulate their data or fetch it from an API.

Data should not be hardcoded despite claiming simulation using numpy

Does the exercise work correctly and does the learner
demonstrate understanding?

Yes No

Dependency Files

Examine the dependency files:

- requirements.txt: Are the required packages listed with appropriate versions?
- pyproject.toml: Is it properly formatted for Poetry?

Check the program's output and code:

- Does the program clearly show the differences between pip and Poetry?
- Does the code demonstrate understanding of dependency management concepts?

Is the dependency management implementation satisfactory?

Yes No

Exercise 2

Exercise 2 - Preliminaries

Navigate to ex2/ directory and examine the files:

Required files:

- oracle.py
- .env.example
- .gitignore

Before starting the test, ask the reviewee to:

1. Show that python-dotenv is declared in their requirements.txt
   (you can carry one over from ex1/)
2. Ask them to create and activate a virtual environment,
   then install dependencies:

   ```
   $> python3 -m venv .venv
   $> source .venv/bin/activate
   $> pip install -r requirements.txt
   ```

The reviewee should perform these steps themselves.
Do not install anything manually as a reviewer.

Did the reviewee add python-dotenv to requirements.txt?
Did the reviewee successfully demonstrate installing
dependencies in a virtual environment?

Yes No

Environment Variables (The Oracle)

Test the environment configuration:

1. Run oracle.py without any environment variables set
2. Create a .env file based on .env.example with test values
3. Run the program again and verify it loads the configuration
4. Test with some environment variables set directly in the shell

The program should:

- Use python-dotenv library to load .env files
- Load configuration from environment variables
- Use .env file for development settings
- Handle missing configuration appropriately

- Show loaded configuration (without exposing real secrets)
- Demonstrate understanding of environment-based configuration

Ask the learner to explain:

- Why environment variables are important for security
- The difference between development and production configuration
- How python-dotenv helps with configuration management
- How their program handles missing configuration

Does the exercise work correctly and does the learner demonstrate understanding?

Yes No

Security Implementation

Check the security implementation:

- .env.example: Contains template values, not real secrets?
- .gitignore: Properly configured to exclude .env files?
- oracle.py: No hardcoded secrets in the source code?
- python-dotenv: Properly used to load environment variables?

Check the program and code quality:

- Does the program demonstrate understanding of environment variables and security best pra
- Does the code show proper use of python-dotenv library?
- Does the code show proper configuration management techniques?

Verify the learner understands:

- Why secrets should never be committed to version control
- How environment variables enable different configurations for different environments
- The role of python-dotenv in loading .env files
- The security implications of configuration management

Is the security and configuration implementation satisfactory?

Yes No

Overall Understanding and Best Practices

Overall Understanding

Assess the learner's overall understanding by asking them to explain:

1. How these three concepts (virtual environments, package management, environment variables) work together in a real data engineering project
2. Why each of these tools is important for professional development
3. What problems would occur without these tools
4. How they would set up a new data engineering project using all three concepts

The learner should demonstrate understanding beyond just making the code work; they should understand the WHY behind these tools and be able to explain the concepts clearly to others.

Does the learner demonstrate comprehensive understanding of the project's learning objectives?

Yes No

Code Quality and Best Practices

Evaluate the overall code quality:

- Is the code written in Python 3.10 or higher?
- Does the code adhere to the flake8 linter standards (no errors)?
- Are type hints REQUIRED for all functions and methods (parameters and return values)?
- Docstrings are NOT required for this module.
- Is the code well-structured and readable?
- Are error handling and edge cases properly managed?
- Do the programs follow Python best practices?

- Is the documentation clear and helpful?

The code should demonstrate not just working functionality, but good software engineering practices that would be expected in a professional environment.

Is the overall code quality satisfactory for a data engineering professional?

Yes No

Ratings

Don’t forget to check the flag corresponding to the defense

Ok Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat

Concerning situation Leaks Forbidden function Can’t support /

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33)
Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2)
Privacy policy (https://profile.intra.42.fr/legal/terms/5)
General term of use of the site (https://profile.intra.42.fr/legal/terms/6)
Rules of pro (https://profile.intra.42